

Code Review Exercise

Hello and thanks so much for taking the time to do the Ferocia Code Review Exercise. This forms part of our Technical Interview.

Technical Interview Process

1. We send you some requirements (below) and an application (attached .zip) that is supposed to meet those requirements.
2. You review the app against the requirements and against your personal view of what good quality code/development look like. More details below.
3. *Optional - If you really dislike how the app is built, you can refactor onto a branch. There won't be any extra credit for this, but if it helps you with the pairing session later you are welcome to.*
4. 90 minute Technical Interview:
    a. 5 minute **meet and greet**.
    b. You give us a 15 minute **walkthrough of your review**. Format is up to you, can talk us through it, show us code, todos, whatever makes most sense to you.
    c. 30 minute **pairing session** where we extend the application. You can do the extension on your branch if you decided you wanted one, or on the main application. We will be looking at how you breakdown requirements, solve problems, and pair with another engineer. **Please come to the interview (in person or online) with the application loaded into your preferred IDE and ready to code.**
    d. 30 minute **technical interview**. We will be asking you questions about your approach to software engineering and specific experiences you have had. There will be no questions that you would need to Google the answer to.

e. 10 minutes to **ask our interviewers questions** about working at Ferocia.

Why are we asking you to do this?

We want to talk to you about code - how you write it, how you think about it, how you review it, how you turn ideas into concrete programs, what makes good code. We want to do this in a way that gives you the chance to show your skills at their finest, in a context that we both understand. This code review exercise is designed to give us common context that we are both "experts" in so that we can have that talk.

What are we asking you to do?

We are a company that creates banking software. Our intern, Gen A. Eye, has built a very simple term deposit calculator that they are very proud of. We need you to review Gen's code before we ship it to production. You find a few things wrong with Gen's code so decide to sit down with Gen (our interview panel) to walk them through your findings. We want you to pitch your review at someone who is relatively new on the job, gets how to code in a basic way but needs you to educate them on what good looks like.

Application functional requirements

Build an application that takes as inputs:

- Start deposit amount (e.g. $10,000)
- Interest rate (e.g. 1.10%)
- Investment term (e.g. 3 years)
- Interest paid (monthly, quarterly, annually, at maturity)

And produces as output:

- Final balance (e.g. $10,330 on the above inputs, interest paid at maturity)

Gen was supposed to check their outputs using this calculator*. They assumed that all proceeds were reinvested into the term deposit for its duration.

Gen's non-functional requirements

- CLI is fine, although they were welcome to add in a basic UI. However they should take basic usability into account - can the user perform the tasks they

need to, and is it clear how and what they need to do and what the application is doing.

- Gen chose their strongest language and wrote what they thought was production-ready code. This is the standard you should measure against when doing your review.
- Gen was allowed to use a code assistant, you can be the judge of how well they did that.

What are we looking for?

- Timeboxing - keep the review to 15 minutes.
- Don’t feel like you have to get through every single problem with the application in that 15 minutes.
- Not looking for a presentation (e.g. no Powerpoint), pretend you are talking to an intern and talk through your review like you are speaking to a colleague.